

Technical Review

Smart i-LAB Zone 5

Robust Desk/Zone 5 occupancy estimation across CV, AIR-1, power, mmWave, SEN55, and live-service gates

[pctiope/smart-i-lab-testbed](#)

May 2026

Technical Review Scope

What is reviewed

- A Zone 5-only pipeline for occupancy probability, live evidence, and model promotion.
- Source, label, feature, model, validation, serving, and deployment contracts.
- Runtime boundaries between collectors, trainer, promoter, FastAPI app, and UI.

What is intentionally excluded

- Populated environment values, camera URLs, API keys, live logs, and generated run metrics.
- Runtime data/model artifacts under `data/`, `model/`, and `logs/`.
- Any claim that dashboard health alone proves a candidate model is promotable.

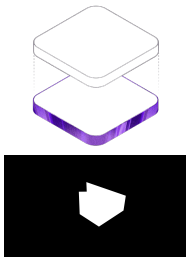
Source-grounded

No secrets

No snapshot metrics

Operational gates

Problem Setting



Tracked dashboard and Desk 5 ROI assets.

Research setting

Desk/Zone 5 is a live lab zone, not a fixed benchmark. Occupancy evidence arrives through CV person counts, AIR-1 environmental readings, smart-plug power, mmWave state, SEN55 air-quality readings, and time.

- Streams can be stale, delayed, partial, or absent while the dashboard remains available.
- CV labels are proxy truth and inherit mask, lighting, and occlusion limits.
- The model must expose probability and readiness without learning missingness as occupancy.

Research Questions

1. Can Zone 5 combine environmental, power, mmWave, SEN55, CV, and time evidence without treating missingness itself as occupancy evidence?
2. Can short-history mmWave recency improve robustness while keeping `zone_occupied` CV-derived?
3. Can validation and promotion gates keep live serving stable while the model contract evolves?
4. Can the same codebase support production user-systemd services and staged Compose checks without mixing ownership?

The answer is framed as a complete system walkthrough: data flow, package structure, model contract, training loop, serving endpoints, deployment boundaries, limitations, and source anchors.

System Scope And Ownership

Package scope

- Package: `zone5_cv_time_features_package`.
- Runtime state roots: `data/`, `model/`, `logs/`.
- Training target: `zone_occupied`.
- Sample cadence: 10-second buckets.
- Model zones: Zone 5 only.

Service ownership

- Collectors own live rows and package-local CSVs.
- Trainer owns stable snapshots and candidate run folders.
- Promoter owns `model/production_run.txt`.
- FastAPI owns serving, readiness, and dashboard payloads.
- UI/Vite shell is separate from model validity.

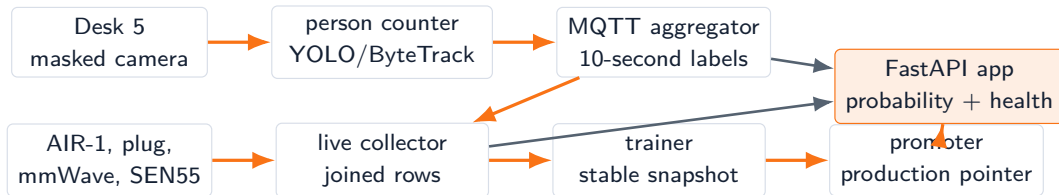
Boundary

Production runtime is user-systemd. Docker Compose is a staging surface and ops/test helper; it does not replace the production trainer or production pointer policy.

Package Structure

Path	Review role
zone5/feature_contract.py	Declares raw features, missing indicators, mmWave recency columns, target, sample interval, and lookback choices.
zone5/build_cv_training_data.py	Joins cleaned feature rows with 10-second CV labels and preserves null labels.
zone5/collect_training_data.py	Live append path for AIR-1, power, mmWave, SEN55, CV labels, and metadata refresh.
zone5/training.py	1D CNN, Optuna search, rolling-calendar CV, blind-test split, final training, and artifact metadata.
zone5/promote_model.py	Promotion gates, smoke loading, positive evidence, strict-fold policy, and same-window non-regression.
web_app/main.py	FastAPI routes, health payload, SSE stream, mask, and MJPEG endpoint.
systemd/user/	Production service and timer declarations for the live stack.

End-To-End Runtime Flow



Collect

Join

Snapshot

Train

Promote

Serve

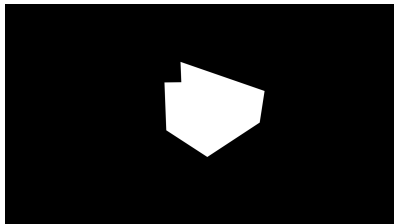
Active Services

Unit	Owned behavior
<code>zone5-person-counter.service</code>	Runs the masked Desk 5 person counter and emits count evidence plus latest annotated frame.
<code>zone5-mqtt-aggregator.service</code>	Converts person-count messages into 10-second CV buckets.
<code>zone5-sen55-collector.service</code>	Buckets SEN55 MQTT readings into package-local air-quality rows.
<code>zone5-live-collector.service</code>	Appends deduplicated joined rows and refreshes training metadata.
<code>zone5-live-app.service</code>	Serves FastAPI dashboard data, health, SSE, MJPEG, mask, and model probability when ready.
<code>zone5-trainer.timer/service</code>	Runs scheduled oneshot retraining and promotion without overlapping active retrains.
<code>zone5-vite-frontend.service</code>	Optional/persistent frontend shell; it does not decide model promotion.

Operational rule

Collection can continue, the app can stay available, and promotion can fail independently. The production pointer is the serving gate.

Person Counter Path



Tracked Desk 5 mask asset.

Source-grounded behavior

- Launcher path: `run_person_counter.sh`.
- Tracker script: packaged masked RTSP person tracker.
- Default tracker family: YOLO with ByteTrack-style tracking.
- Main artifacts: person-count CSV and latest annotated JPEG.
- MQTT count topic is evidence; it is not the final model target until bucketed.

Label chain

Person detections become count messages, then 10-second bucket rows, then the `zone_occupied` target.

CV MQTT Aggregation

Bucket contract

- Time base: 10-second sample boundary.
- Aggregation: median occupancy count per bucket.
- Late-grace handling allows delayed messages before flush.
- Output: package-local CV occupancy CSV.

Target rule

- Positive when median occupancy count reaches the configured threshold.
- Negative when median count is below threshold.
- Null when no CV label exists.
- Null CV buckets are not treated as vacancies.

Proxy truth

10-second rows

Null-preserving

CV-derived target

Sensor Acquisition Paths

Family	Columns	Operational note
AIR-1 Zone 5	Temperature, humidity, CO2, and PM2.5	Core environmental inputs; live collector fetches recent history.
Smart plug	power_s5	Core input for Zone 5 only; not shared with AIR1 all-zones.
mmWave	mmwave_s5	Core input plus recency features; target remains CV-derived.
SEN55	PM, temperature, humidity, VOC, NOx	Optional family; neutral-filled and dropout-tested during training.
Time	Hour/day-of-week cycles	Deterministic cyclic features from timestamp.

No secret material

The deck names column contracts and source paths only. It does not include live API URLs, camera URLs, credentials, or generated logs.

Live Collector Contract

Collector inputs

- Recent AIR-1 Zone 5, power, and mmWave histories.
- Package-local SEN55 bucket CSV.
- Package-local CV label CSV.
- Backfill window for late CV and SEN55 rows.

Collector outputs

- Joined training CSV.
- Joined metadata JSON.
- Deduplicated 10-second rows.
- Rolling CSV retention through the CSV size guard.

The live collector refreshes joined evidence; the trainer snapshots it before fitting so appends do not mutate a run mid-fit.

Feature Contract

Group	Columns	Model input?
AIR-1	temp_s5, rh_s5, co2_s5, pm25_s5	Yes
Power	power_s5	Yes
mmWave raw	mmwave_s5	Yes
mmWave recency	1m/3m/5m recent fractions, minutes since last occupied	Yes
SEN55	PM, temperature, humidity, VOC, NOx fields	Yes, optional family
Time	Hour and day-of-week sin/cos	Yes
Missing indicators	*_missing for raw feature columns	Diagnostics/metadata only
Target	zone_occupied	No, label

Current contract

zone5_mmwave_recency_v1 extends the missingness-decoupled contract: raw sensors stay in the table, while missingness channels are not part of FEATURE_COLUMNS.

Core-Vs-Optional Gates

Core readiness families

- AIR-1 Zone 5 environmental fields.
- `power_s5`.
- `mmwave_s5`.
- Minimum present fractions are stored in artifact metadata and checked during training.

Optional SEN55 family

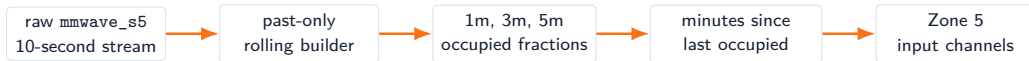
- SEN55 columns are preserved as raw sensors.
- Missing values can be neutral-filled for trainability.
- SEN55 dropout tests whether the model survives that family disappearing.
- SEN55 absence should not become an occupancy proxy.

Keep raw sensors

Gate core inputs

Do not learn missingness

mmWave Recency Without Leakage



- Recency is derived from samples at or before the current timestamp.
- Defaults protect rows with no prior mmWave occupancy.
- mmWave is an input feature and a health signal.
- zone_occupied remains the CV-derived target.

Training Table Semantics

One row means

- A floored 10-second timestamp for Zone 5.
- Latest joined sensor state available for that timestamp.
- CV label if the person-count bucket exists.
- Missing raw sensor values preserved before preprocessing.

One row does not mean

- A camera frame is guaranteed for every sensor row.
- Missing CV labels are negative labels.
- mmWave is target truth.
- A row is promotable evidence before split and coverage checks.

Review implication

The model claim is a table contract plus validation policy, not just a trained checkpoint.

Rolling Window Inputs

Window construction

- Candidate lookbacks: 15, 60, and 180 minutes.
- At 10-second cadence, these are 90, 360, and 1080 rows.
- Windows are built from feature rows only; labels supervise window endpoints.
- Scaling and fill values are learned from training partitions.



Tensor shape

Conceptually: feature channels by time. The CNN learns local temporal patterns before global pooling.

1D CNN Architecture

Stage	Source behavior
Input	FEATURE_COLUMNS channels across the selected lookback window.
Conv blocks	2 or 3 Conv1d blocks; base channels grow by block and cap at 256.
Kernel choices	First and late kernels are searched independently over compact odd sizes.
Normalization	Optional batch normalization after convolution.
Activation	Search over ReLU, GELU, and SiLU.
Pooling	Max-pooling after block 2 or after each late block.
Global pooling	Adaptive average or max pooling to one temporal summary.
Classifier	Dense layer, activation, dropout, final single-logit output.

TunableZoneOccupancyCNN

Conv1d

Adaptive pool

Single binary logit

Hyperparameter And Training Loop

Search space

- Lookback, batch size, learning rate, weight decay.
- Number of convolution blocks, base channels, kernel sizes.
- Activation, normalization, pooling pattern, global pool.
- Dense units, dropout, scheduler, patience.

Training mechanics

- Optimizer: AdamW.
- Class imbalance: positive weight from train labels.
- Schedulers: none, cosine, or reduce-on-plateau.
- Objective: mean validation PR-AUC over eligible folds.
- Sampler: Optuna TPE with a fixed seed path.

No generated results

This review names the loop and gates but does not embed current run IDs or metric values.

Training Snapshot And Pointers



- The trainer snapshots current joined rows under `data/training_snapshots/`.
- The lock prevents overlapping retrains.
- `current_run.txt` is candidate state.
- `production_run.txt` is the live-serving gate.

Validation Split Policy

Offline split discipline

- Latest calendar day is held out as blind-test evidence.
- Earlier eligible dates form one-day rolling validation folds.
- Strict mode requires usable train and validation labels.
- Bootstrap fallback is first-production recovery, not normal replacement.

Lookback viability checks

- A lookback must fit train and validation windows.
- Both-class windows are required unless explicitly in smoke/dev mode.
- Under-covered dates and rejected lookbacks are recorded in metadata.

Blind test

Rolling calendar

Both-class windows

Recorded split policy

Promotion Gates

Candidate quality gates

- Strict/allowed validation mode.
- Progressive strict-fold maturity.
- Blind-test PR-AUC and mean CV PR-AUC thresholds.
- Brier score and log-loss ceilings.
- Positive windows, positive buckets, and positive events.

Operational gates

- Smoke loading for artifact compatibility.
- Same-window non-regression against production when comparable.
- Atomic production pointer update on success.
- Failure returns status without stopping collectors or app serving.

Smoke Loading And Artifact Compatibility

What smoke checks protect

- Target column matches `zone_occupied`.
- Feature columns match the declared contract.
- Legacy missingness feature channels are rejected for current models.
- The production pointer can be read without changing live data.

Why it matters

- Contract evolution is expected.
- A checkpoint alone is not enough to serve safely.
- Serving reloads should fail closed rather than silently mis-score inputs.

Review claim

Promotion is a software contract gate as much as a metric gate.

Serving Loop And Model Reload

Runtime inputs

- Production pointer path.
- Live AIR-1/power/mmWave data source.
- Package-local SEN55 and CV-label tables.
- Configured tick interval, max gap, max age, and threshold.

Runtime outputs

- Latest occupancy probability event.
- History buffer for charting and SSE.
- Health payload with readiness and last-error context.
- MJPEG frame from live or file-backed grabber.

Pointer-pollled

History-sized

Health-first

Stream-capable

FastAPI Endpoint Surface

Endpoint	Purpose
/	Static dashboard shell.
/api/health	Readiness, data-source, model pointer, history, video, mask, and last-error state.
/api/mask.png	Packaged Desk 5 ROI mask when available.
/api/current	Latest inference/history event, or 503 when history is empty.
/api/history?n=	Bounded recent history for charts and diagnosis.
/api/stream	Server-sent events for live dashboard updates.
/api/video.mjpg	MJPEG video stream from RTSP or file-backed annotated frame source.

Interface constraint

The presentation change does not alter public API, schema, services, filenames, or Make targets.

Health And Readiness Semantics

Healthy enough to operate

- The app can answer `/api/health`.
- Video and mask routes can remain useful for diagnosis.
- Collectors can continue appending rows.
- Last error can explain an upstream or pointer blocker.

Not equivalent to

- A fresh production model exists.
- The candidate model passed promotion.
- All upstream sensors are fresh.
- The current dashboard probability is validated on new data.

Readiness is a composable health surface: serving status, source freshness, model pointer state, and error context are distinct signals.

Reproducibility And Deployment Boundary

Production user-systemd

Production owns live collection, app serving, optional frontend, trainer timer, and production pointer state in the user checkout.

- Source deploy restarts live services only.
- Model delivery and training remain separate workflows.
- Runtime state stays out of git.

Docker Compose staging

Compose validates service wiring and staging deploy behavior on separate staging ports and branch scope.

- Compose checks code, config, and image build.
- Compose does not add a production trainer timer.
- Manual proof precedes scheduled automation.

Operational Validity Checks

Code and config evidence

- Python compile checks for package and web app.
- CV ground-truth unit tests.
- Docker Compose config validation and image build.
- Smoke test for model artifact loading.

Runtime evidence

- `/api/health` for readiness and last error.
- Timer/service status for collector and trainer ownership.
- Pointer files for candidate vs production model state.
- Logs inspected only locally; not embedded in the deck.

Why this belongs in research review

The model is only meaningful when data generation, validation, promotion, and live serving preserve the same contract.

Limitations And Threats To Validity

Data and label threats

- CV proxy truth can fail under occlusion, lighting shifts, or mask errors.
- Sensor/API outages can reduce feature coverage or delay joined rows.
- Class imbalance and sparse occupied windows can make metrics unstable.
- Delayed buckets can revise recent rows after first append.

Operational threats

- Upstream latency can stretch prediction cadence.
- Artifact compatibility must be explicit as contracts evolve.
- A healthy dashboard is not proof that a candidate is valid.
- Scheduled retraining must avoid overlapping live fits.

Roadmap

Near-term hardening

- Continue strict rolling-calendar promotion before expanding scheduled automation.
- Track model freshness, source freshness, and label coverage as first-class readiness signals.
- Keep mmWave recency monitored for leakage and stale-input behavior.

Research extensions

- Quantify feature-family ablations under the missingness-decoupled contract.
- Compare recency windows once enough positive evidence accumulates.
- Add reviewer-facing diagnostics that summarize split and coverage policy without exposing runtime secrets.

Source Anchors

Topic	Primary source paths
Pipeline map	PIPELINE.md
Feature and target contract	zone5/feature_contract.py
CV label builder	zone5/build_cv_training_data.py
Live collection	zone5/collect_training_data.py
Model and training	zone5/training.py
Promotion	zone5/promote_model.py
Serving endpoints	web_app/main.py
Deployment boundary	CICD_SYSTEMD.md, CICD_DOCKER_COMPOSE.md

Closing point: Zone 5 treats model contract, labels, validation, and rollout gates as one technical system.